ROS 2 Security and Threat Modeling

Review report and revision plan

Juan Carlos Nevarez · Gh0stlyKn1ght
Security review, editorial direction, and proposed additions
GitHub: @Gh0stlyKn1ght
Review date: **October 20, 2026** · Planning edition 1

Finding: The security material is worth keeping, but it needs a substantial revision before it can serve as a current implementation guide. Preserve the architecture, trust-boundary analysis, and historical case studies. Correct the instructions, distinguish proposed controls from demonstrated controls, and add one reproducible model for a supported deployment.

This report reviews the local `design` repository, branch `gh-pages`, revision `93a415bf928751d37fce6f83215c521658e20c93`. The fork is Gh0stlyKn1ght/design; upstream is ros2/design. All four dedicated security article files matched the live upstream `gh-pages` files byte for byte during this review. These are upstream documentation issues, rather than differences introduced by this fork.

The deliverables include this report, a complete section inventory, an extracted historical reference reader, and a separate landscape figure atlas. The article sources remain unchanged. The reader changes presentation, not the historical technical claims. No robot, network, or middleware runtime was assessed.

1. What security material exists?

There are **four dedicated security articles**, with **6,207 source lines** in total. Their declared modification dates are from 2020–2021. These dates describe the articles' own front matter; they are not release support dates or proof of their last substantive review.

Article	Declared last modified	Assessment
180 · DDS-Security integration	July 2020	Useful foundation; correct interoperability expectations and add tested setup and failure cases.
181 · Access Control Policies	June 2021	Keep least privilege and policy concepts; validate schema and examples against a pinned SROS 2 version.
182 · Security Enclaves	July 2020	Keep identity/context discussion; correct keystore guidance and separate implemented behavior from proposals.
183 · Robotic Systems Threat Model	January 2021	Strong historical threat catalog; replace obsolete deployment assumptions and modernize evidence and risk treatment.

Article 180 covers authentication, access control, cryptography, governance and permissions files, enclave materials, security environment variables, permissive versus enforced startup, and SROS 2 tooling.

Article 181 covers mandatory access control, least privilege, privilege separation, XML validation and transformation, policy composition, enclaves/profiles, topic/service/action privileges, templating, alternative representations, and policy concerns. Its embedded schema and examples are included in the extracted reader.

Article 182 covers participants, namespaces, contexts, keystore organization, enclave lookup, launch proposals, composition, shared process memory, and middleware migration.

Article 183 covers actors, assets, entry points, trust boundaries, generic threats, STRIDE/DREAD tables, adding a robot model, TurtleBot 3 and MARA architectures, platform-specific threats, attack trees, validation strategies, and historical MARA assessment findings. This is a 5,308-line draft, not a short tutorial.

Additional security discussions

The inventory also records every security-related match in the tracked first-party Markdown, HTML, JavaScript, configuration, and XML sources, with file, line, and enclosing heading. The five additional articles with explicit security-related passages are:

File	Relevant section or passage
`articles/160_ros_command_line_arguments.md:250–268`	Security enclave assignment and overrides.
`articles/170_node_to_participant_mapping.md:127–132,159`	Security artifacts, process mapping, discovery topic permissions.
`articles/actions.md:426`	Deterministic action topic names and policy generation.
`articles/020_ros_with_dds.md:84`	Historical description of DDS evolution and security.
`articles/unique_network_flows.md:58`	5G Network Exposure Function API security claim; contextual, not a security design.

The extracted reader includes the enclave argument section, participant security section, and the other brief passages. It also retains the participant-mapping paragraph around line 159.

Security-adjacent site code appears in `README.md:41`, `_includes/header.html`, `_includes/sidebar.html`, `js/login.js`, `js/github.js`, and `js/ros2-design.js`. These concern GitHub OAuth, credential handling, and the old login integration. `index.md` supplies the Security category. Third-party Bootstrap license wording and certificate icon names are keyword noise, not additional security lessons.

Related background worth cross-linking during revision includes topic/service name mapping, discovery, QoS, launch, lifecycle, and intra-process communication. Those topics support security reasoning even where they do not use security terminology.

2. What is outdated or incorrect?

P0 · Correct before teaching implementation

F01 — Wrong keystore location in the enclaves article. Article 182 line 105 tells readers to point `ROS_SECURITY_KEYSTORE` at the `enclaves` directory. Article 180 lines 133–159 and the official Jazzy setup example use the **keystore root**, which contains `enclaves`. Fix the conflicting sentence. Show an absolute enclave path separately. Evidence: local articles and official Jazzy setup source.

F02 — STRIDE expansion has a factual error. Article 183 lines 420–422 expands the “I” as “Integrity.” It should be **Information Disclosure**. Its table already uses the correct category. Correct the prose and retain a consistent six-category legend. Evidence: Microsoft STRIDE definitions.

F03 — Secure cross-vendor communication is overstated. Article 180 repeats that mandated builtin plugins make security work across vendors with minimal effort. The current Jazzy setup source explicitly says secure communication between vendors is not supported. Replace the promise with a versioned support statement and record actual combinations tested. Standards compatibility is not a test result. Evidence: official Jazzy setup source, alternate-middleware section.

F04 — Security enabled does not describe the policy. Article 183's “SROS Enabled?” field means whether an attack remains possible while SROS is enabled; it is easy to read the checkmark in the opposite way. It also compresses important configuration differences. Replace it with explicit prerequisites: authentication, authorization, governance protection, credentials, transport, and endpoint behavior. Keep the historical legend in the extracted reader.

F05 — Identity, signing, and hardware protection are mixed together. Around article 183 lines 2522–2546 and 4030–4054, recommendations include binary signing by the Permissions CA, certificate revocation, TPM storage, and “secure enclave” protection. Rework these as separate controls with named owners and implementation requirements. An SROS 2 enclave organizes security artifacts; it does not provide a hardware trusted execution environment or prevent a compromised process from accessing its readable keys. Separate software-update signing from DDS identity/permissions administration. Mark hardware-key integration and revocation behavior as vendor/version-dependent until tested. This is an editorial design recommendation based on the local claims, not a verified product capability.

P1 · Replace the operational baseline

F06 — The reference deployments are historical. TurtleBot 3 lists Raspbian Stretch, Ubuntu 18.04, and ROS 2 Crystal. MARA also lists Crystal. Crystal reached end of life in December 2019. Keep those case studies explicitly dated; create a new baseline with exact OS, architecture, ROS distribution, RMW, middleware/security library versions, firmware, and topology. The current official release table lists Lyrical through May 2031 and Jazzy through May 2029; choose based on hardware/package support and verify it in the target lab. Kilted is listed through December 2026, making it a poor default for a long-lived new edition. Evidence: official release table source.

F07 — The DDS standard reference needs a compatibility note. Articles 180/182 cite DDS Security 1.1. OMG now lists **DDS Security 1.2**, adopted February 2026. Link both the historical basis and the current standard. Do not simply replace the version number or section numbers: first check the implementation's supported features and relevant specification changes. Evidence: OMG DDS Security 1.2.

F08 — DDS-specific controls are described too broadly for all ROS 2. Add a transport scope statement. The current `rmw_zenoh` project documents a different security configuration path using `zenoh_security_tools`. DDS governance and SROS 2 examples must be labeled for supported DDS RMWs. Bridges create additional trust boundaries and require their own policies. Evidence: official rmw_zenoh README, Security.

F09 — Risk scores and mitigations lack current evidence. The draft uses DREAD totals and historical RVSS findings. Keep original scores as historical annotations. For the new model, record likelihood rationale, operational exposure, safety/privacy/availability impact, owner, treatment, residual risk, review date, and evidence. Use CVSS only for specific vulnerabilities where sufficient facts exist; distinguish vulnerability severity from deployment risk. FIRST's CVSS 4.0 specification includes safety considerations, but a score does not replace a robot safety assessment. Evidence: local scoring sections and FIRST CVSS 4.0.

F10 — Proposed mitigations look like completed work. AWS feasibility notes, future validation strategies, and statements such as “Risk is mitigated” require qualification. Label each statement **proposed**, **implemented**, **tested**, or **historical**, with evidence where available. MARA findings belong to the specific H-ROS assessment described in the source, not all current ROS 2 robots. Do not infer that old findings remain exploitable or have been fixed.

P2 · Review the inherited website separately

`js/login.js:23` constructs an HTTP request for an OAuth token exchange. The README also describes an HTTP gatekeeper and a free Heroku instance. These are visible code/documentation concerns. Runtime reachability, deployment, response contents, and practical exploitability were not tested. If the fork will be hosted, remove the unused login feature or modernize its transport and credential flow; review dependency versions and permissions at the same time. This website work is separate from securing a robot.

3. What should we add?

The old threat model already discusses physical access, firmware, updates, cloud credentials, secure boot, and isolation. Expand these into testable deployment guidance rather than presenting them as entirely absent topics.

1. **A concrete system model.** Name the robot and mission, operators, maintenance paths, network exposure, data retention, and what is outside scope. Draw data flows across host, process, device, radio/network, cloud, and safety-controller boundaries.
2. **A layered security explanation.** ROS 2 is middleware, not the host OS. Reachability and discovery do not grant authorization. Node names are not authenticated identities. Shared process memory remains a shared trust boundary even when contexts use different credentials.
3. **A current SROS 2 walkthrough.** Explain keystore root, signing authority custody, per-enclave runtime keys, permissions generation, governance choices, and enforcement. Show least-privilege behavior for topics, services, actions, parameters, lifecycle, `/rosout`, and discovery metadata using the chosen implementation.
4. **Credential operations.** Document provisioning, expiration, renewal, key compromise response, policy replacement, signing-key backup, clock dependencies, and recovery. Ship only required runtime material to each robot; retain signing authority keys separately.

5. **Host and supply-chain controls.** Add service accounts, filesystem permissions, constrained capabilities, network rules, signed updates, rollback strategy, dependency inventory/SBOM, provenance, patch cadence, and advisory review. Identify which control stops which threat.
6. **Availability and physical safety.** Include command timeout, stale/replayed input handling, bounded queues/resources, degraded operation, watchdog behavior, independent stop controls, and safe recovery. Authentication alone does not establish that a command is appropriate or safe.
7. **Bridges and non-DDS links.** Account for browser dashboards, remote access, cloud telemetry, ROS bridges, microcontroller/agent links, serial/CAN, and Zenoh where present. State where encryption and authentication terminate.
8. **Observable validation.** Give every threat an ID and every control an owner. Capture successful authorized behavior and denied behavior, sanitized logs, exact versions, and unresolved limitations. Keep proposed, simulated, bench-tested, and physical-robot results distinct.
9. **Optional AI/perception appendix.** If the actual robot uses learned perception or language-model tools, add sensor deception, untrusted model inputs, tool permissions, and command validation. Do not expand the baseline around capabilities the robot does not have.

4. Proposed handbook structure

Chapter	Reuse	New contribution
1 · Scope and prerequisites	Threat-model scope and terminology	Exact supported deployment, objectives, mission and exclusions.
2 · Robot architecture and trust boundaries	Actors/assets/entry points	Current diagram and evidence register.
3 · Threat modeling method	STRIDE and attack trees	Correct legend, explicit risk rationale and threat IDs.
4 · DDS security and transport scope	Article 180	Version matrix, governance, interoperability limits.
5 · Enclaves and permissions	Articles 181/182	Correct paths, tested policy and launch examples.
6 · Host, network, credentials and updates	Existing mitigations	Operations procedures and recovery.
7 · Validation exercise	Existing validation strategy	Bounded lab, negative checks and evidence artifacts.
8 · Historical cases	TurtleBot 3 and MARA	Clear dates, scope and limits on reuse of results.
Appendices	Schemas, references, diagrams	Threat records, glossary, source/ change register.

Each teaching chapter should include an objective, prerequisite, worked example, bounded exercise or artifact, explain-back question, common misconception, limitation, sources, and next step. Avoid completion claims based only on reading.

First worked example

Use a non-actuating talker/listener system as the first lab. Record the selected ROS distribution, DDS RMW and security libraries before running it. Create separate enclaves and a restrictive policy for a known topic. Test authorized messaging, missing credentials, an untrusted identity, and a remapped topic outside the policy. Capture outcomes and failure logs. The official Jazzy access-control tutorial provides a starting point for policy signing and remap rejection; adapt its dates and examples to the lab.

The artifact should be a small architecture diagram, threat record, policy, version manifest, and evidence log. The learner explains why an authenticated participant can still be denied access, and why encrypting traffic does not stop a compromised authorized process. This proposed lab has not been run in this review.

5. Printing and PDF plan

The checkout has **no local PDF, PowerPoint, or slide source files**. Its material is Markdown, embedded HTML, schema/example XML, and raster diagrams. The landing page links external historical presentation PDFs; those are outside this extraction. This means the observed problem here is wide article content, rather than a confirmed defect in a particular slide file.

The generic threat table has **27 logical columns**. The TurtleBot and MARA tables have **11 and 14**. They combine fixed widths, rowspans, large rotated headings, and inconsistent section colspans. A landscape print setting alone will not make these readable on ordinary paper.

Implemented in this planning package:

- A US Letter portrait report and reference reader, with white backgrounds, black text, page numbers, source credits, and linked contents.
- Wide threat matrices converted into vertical records. Every supplied original field is retained, including explicit historical symbols and inherited rowspan values. Malformed source rows with missing or extra cells are retained in source order and explicitly flagged for unresolved alignment. No numeric score is silently recalculated.
- A separate, uniformly landscape figure atlas for the threat-model images. It can be printed as a separate job without mixed-orientation duplex problems. Enlarging the original rasters does not restore missing detail.
- Offline HTML companions and a repeatable Python build. The source inventory records paths, headings, line ranges, keyword matches, and hashes.

Print settings: Use Letter paper and “Actual size” or 100%. For the portrait files, select portrait and long-edge duplex if desired. Print the atlas separately in landscape; short-edge duplex normally keeps landscape pages upright. Printer drivers differ, so confirm the duplex preview before printing a large batch. Disable browser-added headers and footers when printing the HTML; the PDFs already have pagination. A4 is a future alternate build if requested.

For the final revised handbook, redraw the densest figures as editable vector diagrams, split complex attack trees into labeled panels, and keep the threat records as the primary paper format. Preserve the source matrices in machine-readable form for comparison. Add a one-page-per-threat option only if we need worksheets; it would use substantially more paper.

6. Credit and provenance

Use the following cover credit:

Juan Carlos Nevarez · Gh0stlyKn1ght
Security review, adaptation, and additional material
github.com/Gh0stlyKn1ght

The GitHub spelling follows this fork's actual remote owner. It differs from the spellings in the request; the display name can be changed without changing the destination link.

Preserve original article authors next to each extracted article:

- DDS-Security integration: Kyle Fazzari.
- Access Control Policies: Ruffin White and Kyle Fazzari.
- Security Enclaves: Ruffin White and Mikael Arguedas.
- Robotic Systems Threat Model: Thomas Moulard, Juan Hortala, Xabi Perez, Gorka Olalde, Borja Erice, Odei Olalde, and David Mayoral.

The site footer states that design documents are **CC BY 3.0 except where otherwise noted**, with Open Source Robotics Foundation attribution. The repository also contains an **Apache 2.0 LICENSE**. Preserve both notices and their context; do not relabel the entire collection under a single newly chosen license. The extracted reader identifies the original sources and presentation changes, includes the original license text as an appendix, and retains linked author credits. The CC BY 3.0 deed calls for attribution, a license link, and an indication of changes. The handbook should not imply upstream endorsement.

7. Work plan and acceptance criteria

Phase	Work	Acceptance criterion
1 · Inventory and print baseline	Extract, attribute, reflow matrices, generate report and reader.	Every dedicated security article and explicit supporting passage accounted for; no clipped print content. Delivered with this review.
2 · Correct known issues	Fix F01–F05 and label historical/ proposed statements.	Source-level changes reviewed against primary sources; old and new claims are traceable.
3 · Choose one current baseline	Select supported OS/ROS/RMW/ robot combination.	Exact versions and topology recorded; dependencies available on the target hardware.
4 · Author and run the first lab	Keystore, policy, positive/negative cases, evidence.	Authorized behavior works; denied cases fail as intended; logs and limitations included.
5 · Expand the threat model	Add operational controls and a current robot case.	Each threat links to an asset, boundary, control, owner, evidence, and residual risk.
6 · Publish the revised handbook		

Phase	Work	Acceptance criterion
	Editorial review, vector figures, PDF accessibility/print review.	Credits, sources, changes, fonts, pagination, links and paper size verified.

Recommended next step: Correct the known documentation defects and build the talker/listener chapter against one chosen DDS implementation. Use that small verified example to establish the format before rewriting both historical robot models.

8. Evidence and limits

The inventory is generated from tracked repository sources. Live upstream byte comparisons verified the four dedicated article files; this was not a full branch parity check. Primary references were reviewed on October 20, 2026. The ROS documentation website returned an access-denied page, so the official GitHub source files were used instead. The Jazzy documentation branch resolved to `03f26be23292f1b8523b3e4f807f3f123ed5b12a`; the Rolling branch resolved to `0d0c2c2d8c51a3f240bafe0c010e71c1d3d72125` during this review.

This is a documentation review and planning package. It is not a live vulnerability assessment, compliance determination, deployment, ROS runtime test, or proof of a physical robot's safety. No current CVE list or middleware patch audit was performed. The final handbook still needs implementation validation against its selected versions.

Primary references used for the modernization assessment:

- Official ROS release table.
- Official Jazzy security setup.
- Official Jazzy access controls.
- OMG DDS Security 1.2.
- Microsoft STRIDE definitions.
- Official rmw_zenoh security overview.
- FIRST CVSS 4.0 specification.
- Creative Commons Attribution 3.0 deed.